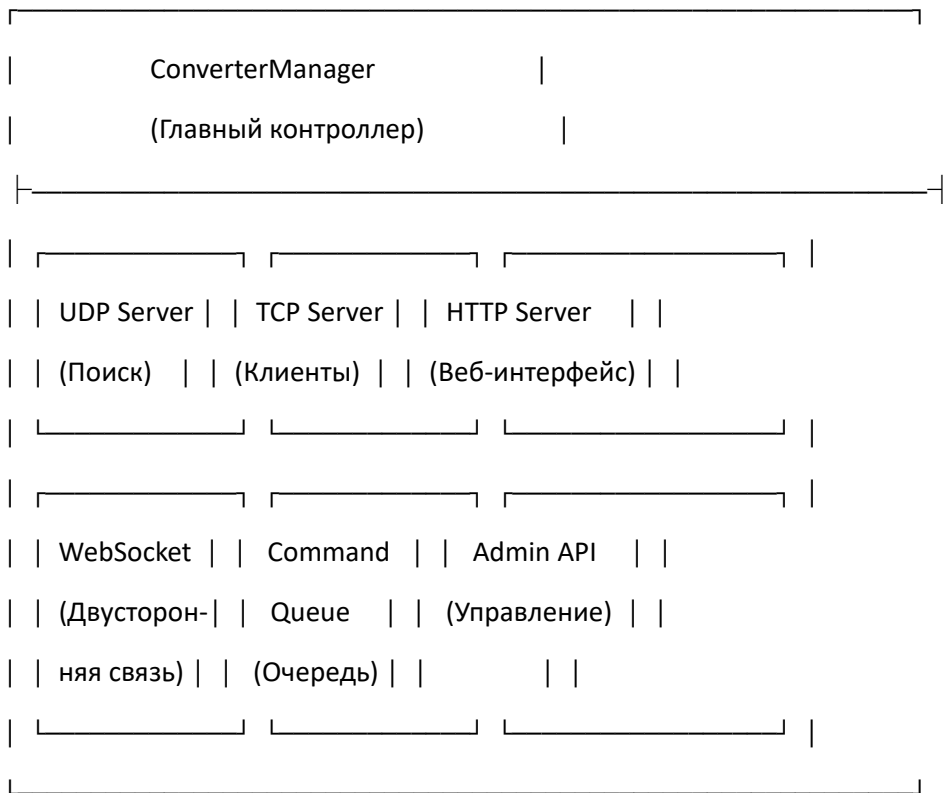


1. Общая архитектура

1.1 Структура приложения

Приложение построено по модульному принципу и состоит из следующих основных компонентов:

text



1.2 Основные компоненты

Компонент	Файл	Описание
ConverterManager	server.js	Главный класс управления
CommandQueue	server.js	Класс очереди команд
UDP Server	server.js	Поиск конвертеров
TCP Server	server.js	Прием клиентских подключений
HTTP Server	server.js	Веб-интерфейс

Компонент	Файл	Описание
WebSocket	server.js	Двусторонняя связь
Admin API	server.js	REST-like API

2. Классы и объекты

2.1 ConverterManager

Главный класс, управляющий всем приложением.

Свойства

javascript

```
class ConverterManager {
  constructor() {
    // Хранилище конвертеров

    this.converters = new Map();    // ip -> data

    this.socketConnections = new Map(); // ip -> socket

    this.commandQueues = new Map();    // ip -> CommandQueue


    // Серверы

    this.udpServer = null;           // UDP сокет
    this.tcpServer = null;           // TCP сервер
    this.httpServer = null;          // HTTP сервер
    this.wsServer = null;            // WebSocket сервер


    // Настройки

    this.logEnabled = true;           // Включить логи
    this.udpBroadcastEnabled = true;  // Включить UDP поиск
    this.broadcastInterval = null;    // Интервал UDP запросов
    this.clients = new Set();          // WebSocket клиенты
  }
}
```

Методы

Категория	Метод	Описание
UDP	initUDPServer()	Инициализация UDP сервера
	startBroadcast()	Запуск широковещательных запросов
	stopBroadcast()	Остановка широковещательных запросов
	broadcastDiscovery()	Отправка UDP запроса
	handleUDPResponse()	Обработка UDP ответа
	parseUDPResponse()	Парсинг UDP ответа
	updateConverterInfo()	Обновление информации о конвертере
TCP	initTCPServer()	Инициализация TCP сервера
	connectToConverter()	Подключение к конвертеру
	disconnectFromConverter()	Отключение от конвертера
	removeConverter()	Удаление конвертера
Управление	setBaudrate()	Установка скорости
	sendRawCommand()	Отправка произвольной команды
HTTP	initHTTPServer()	Инициализация HTTP сервера
	serveFile()	Отдача статического файла
	handleAPI()	Обработка API запросов
	processCommand()	Выполнение команды
WebSocket	initWebSocket()	Инициализация WebSocket

Категория	Метод	Описание
	handleWebSocketMessage()	Обработка WebSocket сообщений
	broadcastConverterUpdate()	Отправка обновления всем клиентам
	broadcastConverterData()	Отправка данных всем клиентам

Структура объекта конвертера

javascript

```
{
  ip: '192.168.1.89',    // IP адрес
  model: 'Z397-WEB-SW', // Модель
  capabilities: 'VCP',   // Возможности
  firmware: '4.0.13',    // Версия прошивки
  serialNumber: '48424',  // Серийный номер
  port1: '1000',          // TCP порт линии 1
  port2: '1001',          // TCP порт линии 2
  conn1: '192.168.1.61', // Подключение линии 1
  conn2: '0.0.0.0',       // Подключение линии 2
  locked: false,          // Состояние LOCK
  status: 'connected',    // Текущий статус
  mode: 'server',         // Режим работы
  lastSeen: 1783544007002, // Время последнего контакта
  baudrate: 230400        // Текущая скорость
}
```

2.2 CommandQueue

Класс очереди команд для организации последовательного выполнения запросов.

Свойства

javascript

```
class CommandQueue {
  constructor(sendFunction) {
    this.sendFunction = sendFunction; // Функция отправки
  }
}
```

```

    this.mainQueue = new Set();      // Основная очередь
    this.operationalQueue = new Set(); // Оперативная очередь
    this.isProcessing = false;      // Флаг обработки
    this.postResponseDelay = 100;    // Пауза после ответа
    this.currentCommand = null;      // Текущая команда
    this.listeners = new Set();      // Слушатели ответов
  }
}

```

Методы

Метод	Описание
add()	Добавление команды в очередь
processQueue()	Обработка очереди
waitForResponse()	Ожидание ответа с таймаутом
listener()	Передача данных слушателям

Структура команды

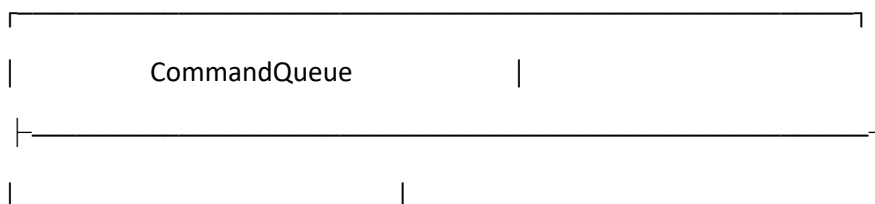
```

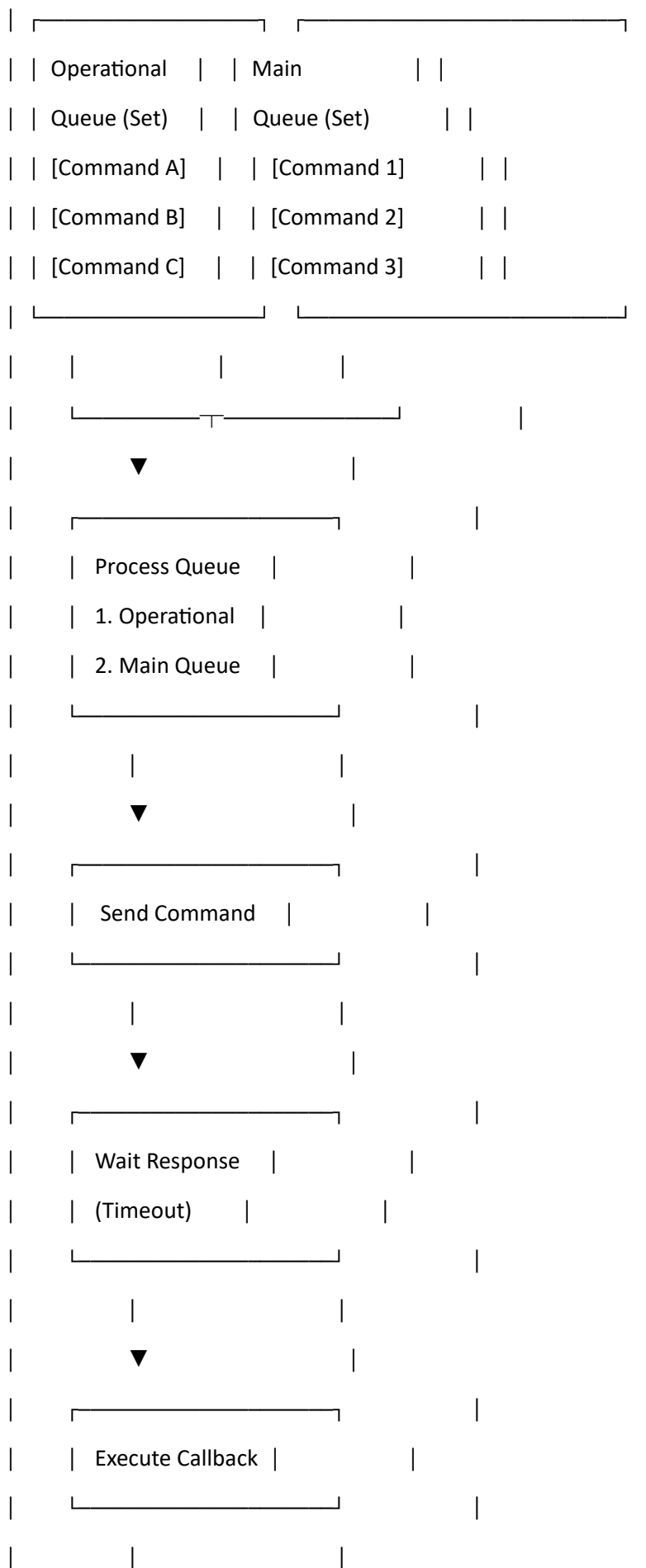
javascript
{
  generator: function,    // Функция-генератор команды
  callback: function,     // Callback для ответа
  timeout: 3000,          // Таймаут ожидания
  timestamp: Date.now(),  // Время создания
  id: 'abc123'            // Уникальный идентификатор
}

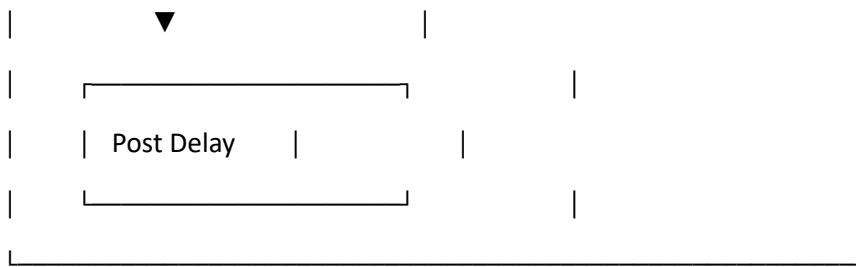
```

Логика работы очереди

text







2.3 Admin API

Объект с методами для управления конвертерами.

Структура

javascript

```
adminAPI = {  
  getConverters: async function(params) { ... },  
  getPages: async function(params) { ... },  
  connectConverter: async function(params) { ... },  
  disconnectConverter: async function(params) { ... },  
  setBaudrate: async function(params) { ... },  
  getConverterInfo: async function(params) { ... },  
  removeConverter: async function(params) { ... },  
  toggleLog: async function(params) { ... },  
  toggleUdp: async function(params) { ... }  
}
```

Параметры методов

Метод	Параметры	Возвращает
getConverters	{}	[converter, ...]
getPages	{}	[{path, title, description}]
connectConverter	{ip}	{message}
disconnectConverter	{ip}	{message}
setBaudrate	{ip, baudrate}	{message}

Метод	Параметры	Возвращает
getConverterInfo	{ip}	{info, response}
removeConverter	{ip}	{message}
toggleLog	{}	{logEnabled}
toggleUdp	{}	{udpBroadcastEnabled}

3. Серверы

3.1 UDP Server (Поиск)

Настройки

javascript

const UDP_PORT = 9000;

const BROADCAST_ADDR = '255.255.255.255';

Протокол поиска

Запрос:

text

SEEK Z397IP

Ответ:

text

Z397-WEB-SW:4.0.13 SN48424 L1_Port:1000 L2_Port:1001 L1_Conn:0.0.0.0 L2_Conn:0.0.0.0 Lock:0

Параметры ответа:

Параметр	Описание
Z397-WEB-SW	Модель устройства
SN	Серийный номер
L1_Port	TCP порт первой линии
L2_Port	TCP порт второй линии

Параметр	Описание
L1_Conn	IP подключения первой линии
L2_Conn	IP подключения второй линии
Lock	Состояние переключателя LOCK

Логика работы

javascript

1. Каждые 3 секунды отправляется широковещательный запрос
2. Конвертеры отвечают UDP пакетом
3. Ответ парсится и сохраняется в Map конвертеров
4. Если конвертер свободен, инициируется TCP подключение

3.2 TCP Server (Клиенты)

Настройки

javascript

```
const TCP_PORT = 9001;
```

Логика работы

javascript

1. Сервер слушает порт 9001
2. Конвертеры в режиме клиента подключаются к серверу
3. Для каждого подключения создается CommandQueue
4. Все данные от конвертера передаются в очередь
5. Команды отправляются через очередь

3.3 HTTP Server (Веб-интерфейс)

Настройки

javascript

```
const HTTP_PORT = 8080;
```

Маршруты

URL	Метод	Описание
/	GET	Главная страница
/index.html	GET	Главная страница
/pages/admin.html	GET	Страница управления
/pages/*	GET	Статические страницы
/stands/*	GET	JSON стендов
/cmd/*	GET	JSON команд
/lists/*	GET	JSON списков
/api	POST	API запросы
/test	GET	Проверка работы

Обработка API

javascript

1. Принимает POST запросы на /api
2. Парсит JSON тело
3. Извлекает {api, cmd, data}
4. Вызывает соответствующий метод adminAPI
5. Возвращает результат в JSON

3.4 WebSocket

Настройки

javascript

ws://[HOST]:8080

Типы сообщений

От сервера:

Тип	Данные	Описание
converter_list	[[ip, status, mode, ...]]	Список конвертеров
converter_update	{ip, data}	Обновление конвертера
converter_data	{ip, data}	Данные от конвертера

От клиента:

javascript

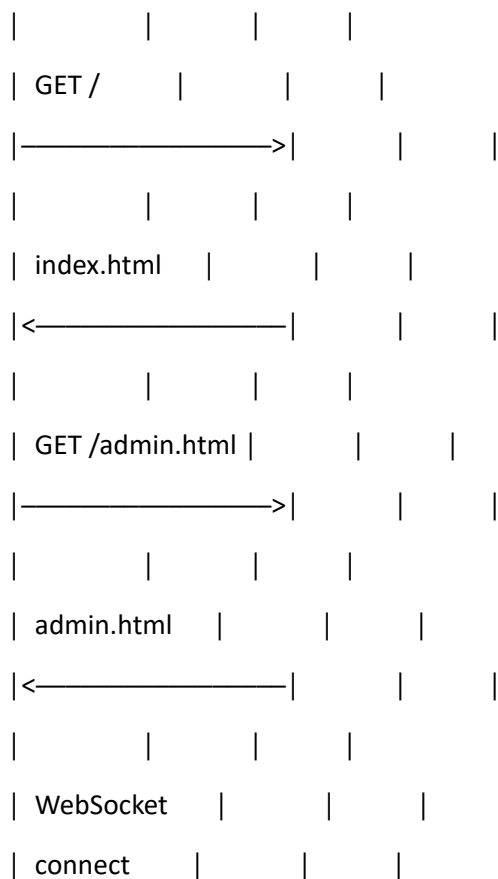
```
{
  action: 'getConverters', // Или другая команда
  params: { ip: '...' } // Параметры
}
```

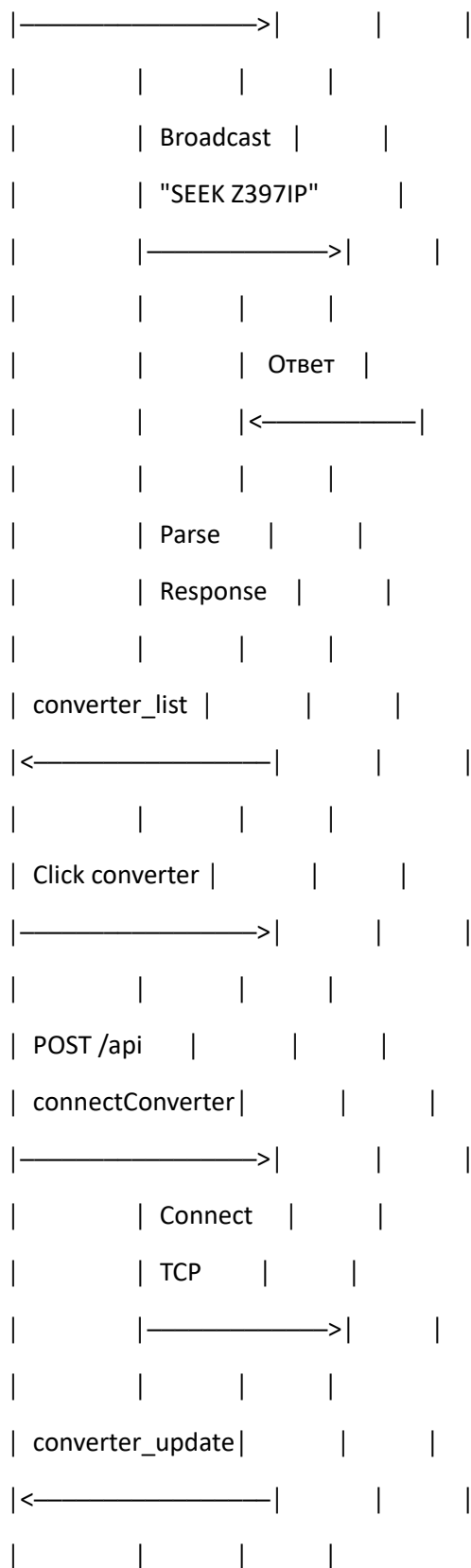
4. Логика работы

4.1 Диаграмма последовательности (UDP поиск)

text

Client (Browser) HTTP Server UDP Server Converter



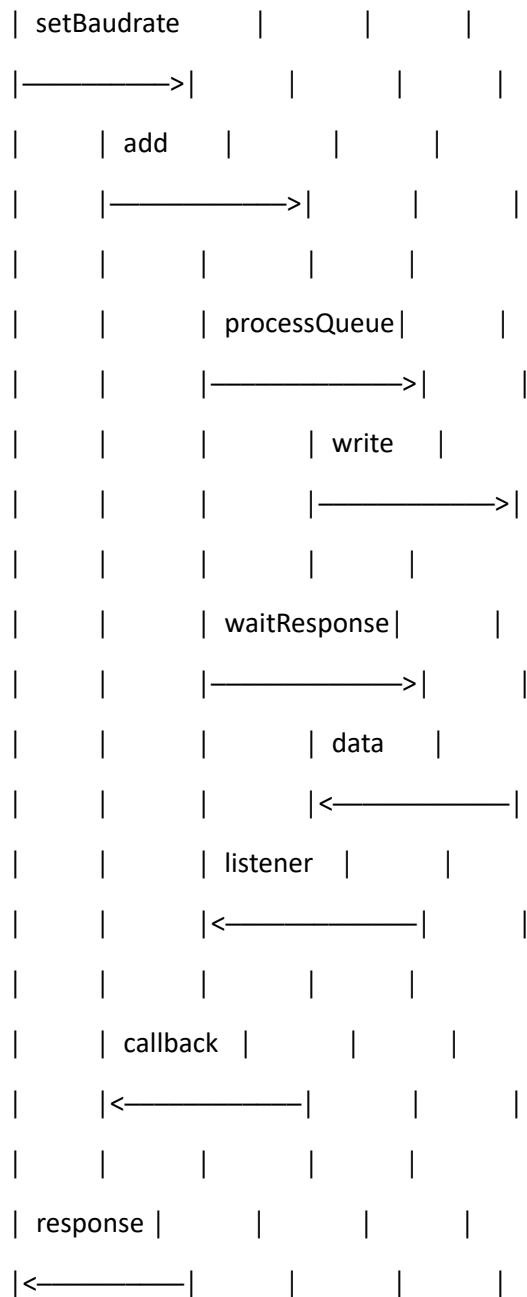


4.2 Диаграмма последовательности (Команды)

text

Client WebSocket CommandQueue TCP Socket Converter





4.3 Логика очереди команд

javascript

// 1. Добавление команды

`queue.add('operational', // Тип очереди (operational/main)`

`() => commandData, // Генератор команды`

`(err, response) => { // Callback`

`if (err) console.error(err);`

`else processResponse(response);`

`},`

`3000 // Таймаут`

```
);
```

```
// 2. Обработка очереди
```

```
while (operationalQueue.size > 0 || mainQueue.size > 0) {
```

```
    // Сначала оперативные команды
```

```
    if (operationalQueue.size > 0) {
```

```
        command = operationalQueue.pop();
```

```
    } else {
```

```
        command = mainQueue.pop();
```

```
    }
```

```
// Отправка команды
```

```
sendFunction(command.generator());
```

```
// Ожидание ответа с таймаутом
```

```
response = await waitForResponse(command.timeout);
```

```
// Выполнение callback
```

```
if (response) {
```

```
    command.callback(null, response);
```

```
} else {
```

```
    command.callback(new Error('Timeout'), null);
```

```
}
```

```
// Пауза перед следующей командой
```

```
await delay(postResponseDelay);
```

```
}
```

4.4 Формат команд NVT

Установка скорости (230400 - ADVANCED режим):

text

FF FA 2C 01 00 03 84 00 FF F0

Структура:

text

IAC SB COM_PORT_OPTION SET_BAUDRATE [4 байта скорости] IAC SE

FF FA 2C 01 [00 03 84 00] FF F0

Скорости в HEX:

Скорость	HEX
230400	00 03 84 00
115200	00 01 C2 00
38400	00 00 96 00
19200	00 00 4B 00
9600	00 00 25 80

5. Форматы данных

5.1 Формат API запроса

json

```
{
  "api": "admin",
  "cmd": "getConverters",
  "data": {}
}
```

5.2 Формат API ответа (успех)

json

```
{
  "success": true,
  "data": { ... }
}
```

5.3 Формат API ответа (ошибка)

json

```
{
```

```
"error": "Сообщение об ошибке"
}
```

5.4 Формат WebSocket сообщения (от сервера)

```
json
{
  "type": "converter_update",
  "ip": "192.168.1.89",
  "data": {
    "status": "connected",
    "mode": "server",
    "baudrate": 230400
  }
}
```

5.5 Формат WebSocket сообщения (от клиента)

```
json
{
  "action": "setBaudrate",
  "params": {
    "ip": "192.168.1.89",
    "baudrate": 230400
  }
}
```

6. Обработка ошибок

6.1 Типы ошибок

Ошибка	Причина	Решение
Timeout	Нет ответа от конвертера	Проверить соединение, увеличить таймаут
Connection refused	Конвертер не слушает порт	Проверить настройки порта
Invalid JSON	Неверный формат запроса	Проверить синтаксис JSON

Ошибка	Причина	Решение
API not found	Неверное имя API	Использовать 'admin'
Command not found	Неверная команда	Проверить список команд
Converter not found	Конвертер не в базе	Выполнить поиск сначала

6.2 Логирование ошибок

javascript

```
logError(message) {
    const timestamp = new Date().toLocaleTimeString();
    console.error(`[${timestamp}] ❌ ${message}`);
}
```

// Пример использования

```
this.logError(`Ошибка подключения к ${ip}: ${err.message}`);
```

7. Расширение функциональности

7.1 Добавление новой страницы

1. Создать HTML файл в /pages
2. Добавить тег <title>
3. Использовать WebSocket и API

7.2 Добавление новой команды в API

javascript

```
adminAPI = {
    // ... существующие методы

    newCommand: async function(params) {
        // Логика команды
        return { result: 'success' };
    }
};
```

7.3 Добавление нового типа сообщения WebSocket

```
javascript
// В методе handleWebSocketMessage
switch(data.type) {
    case 'new_type':
        // Обработка нового типа
        break;
}
```

8. Конфигурация

8.1 Настройки сервера

```
javascript
const CONFIG = {
    udpPort: 9000,        // UDP порт для поиска
    tcpPort: 9001,        // TCP порт для конвертеров
    httpPort: 8080,       // HTTP порт веб-интерфейса
    broadcastAddr: '255.255.255.255',
    defaultBaudrate: 230400, // Скорость по умолчанию
    responseTimeout: 3000, // Таймаут ответа (мс)
    postResponseDelay: 100 // Задержка между командами (мс)
};
```

8.2 Переменные окружения (планируемые)

```
javascript
const CONFIG = {
    udpPort: process.env.UDP_PORT || 9000,
    tcpPort: process.env.TCP_PORT || 9001,
    httpPort: process.env.HTTP_PORT || 8080,
    defaultBaudrate: process.env.DEFAULT_BAUDRATE || 230400,
    responseTimeout: process.env.RESPONSE_TIMEOUT || 3000
};
```

9. Зависимости

9.1 Основные модули

Модуль	Версия	Назначение
ws	^8.14.2	WebSocket сервер
net	(встроенный)	TCP сервер
dgram	(встроенный)	UDP сервер
http	(встроенный)	HTTP сервер
fs	(встроенный)	Файловая система
path	(встроенный)	Работа с путями
os	(встроенный)	Системная информация

9.2 Установка

bash

npm install ws

10. Тестирование

10.1 Тестовый скрипт UDP

javascript

// test_udp.js

```
const dgram = require('dgram');
```

```
const client = dgram.createSocket('udp4');
```

```
client.on('message', (msg, rinfo) => {
```

```
  console.log(`Ответ от ${rinfo.address}: ${msg.toString()}`);
```

```
});
```

```
client.bind(() => {
```

```
  client.setBroadcast(true);
```

```
  const message = Buffer.from('SEEK Z397IP');
```

```
  client.send(message, 0, message.length, 9000, '255.255.255.255');
```

```
});
```

10.2 Тестовый скрипт API

bash

```
curl -X POST http://localhost:8080/api \  
-H "Content-Type: application/json" \  
-d '{"api":"admin","cmd":"getConverters","data":{}}'
```

11. Безопасность

11.1 Текущие меры

- CORS заголовки для ограничения доступа
- Валидация входящих данных
- Обработка ошибок
- Логирование подозрительной активности

11.2 Планируемые улучшения

- JWT аутентификация
 - HTTPS поддержка
 - Rate limiting
 - IP фильтрация
 - Валидация входных данных
-

12. Производительность

12.1 Оптимизация

- Использование Map и Set для быстрого доступа
- Асинхронная обработка запросов
- Очередь команд для предотвращения конфликтов
- WebSocket для уменьшения нагрузки

12.2 Мониторинг

javascript

// Добавить в логи

```
console.log(`Конвертеров: ${this.converters.size}`);
```

```
console.log(`Соединений: ${this.socketConnections.size}`);
```

```
console.log(`WebSocket клиентов: ${this.clients.size}`);
```

13. Планы развития

13.1 Ближайшие задачи

1. ☒ UDP поиск конвертеров
2. ☒ TCP подключение
3. ☒ Веб-интерфейс
4. ☐ Автоматическое переключение в ADVANCED режим
5. ☐ Сохранение конфигурации в JSON
6. ☐ Групповые операции
7. ☐ Мониторинг и статистика

13.2 Долгосрочные планы

- Автоматическое обновление прошивки
- Графики и статистика
- Интеграция с MQTT
- Докеризация
- Поддержка нескольких типов конвертеров
- Расширенная система логирования

Версия документации: 1.0.0

Дата: 2024

Автор: Команда разработки

Документация обновляется по мере развития проекта.